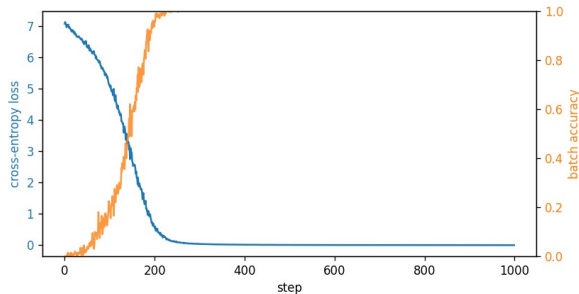


Toy memorisation

Setup: 1-layer transformer, $d_{\text{model}}=32$, $d_{\text{mlp}}=128$, $n_{\text{heads}}=4$, $n_{\text{pairs}}=1024$, 1000 steps, input [BOS, key], target = value token.

R1.1 / R1.2: Baseline trains cleanly; d_{mlp} is not the bottleneck.

Baseline: 1024 pairs, $d_{\text{mlp}}=128$, $d_{\text{model}}=32$

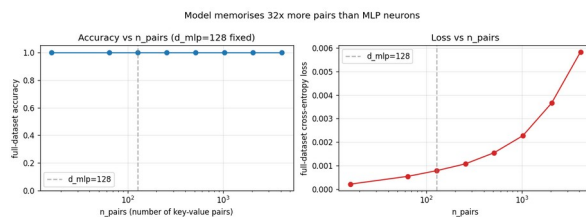


1.1 Baseline curve (loss→0, acc→1).

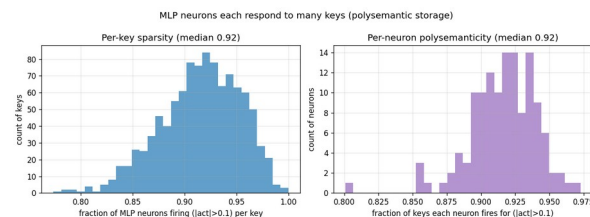
Memorisation succeeds quickly and is robust to large reductions in d_{mlp} .

The memorisation success fastly

R1.3 (Q1): Memorises 32× more pairs than MLP neurons. / R1.4: Compressed storage pattern.



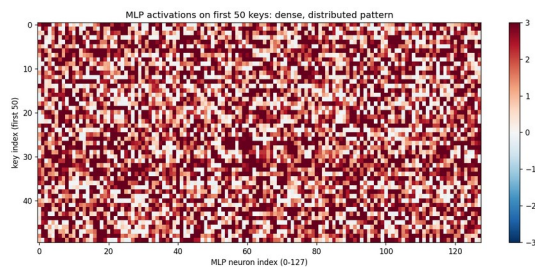
1.3 n_{pairs} sweep at $d_{\text{mlp}}=128$ fixed.



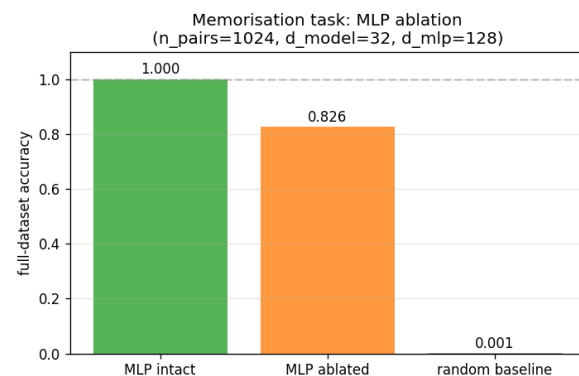
1.4 Each neuron fires for many keys; each key uses many neurons.

Q1 answer: yes — 4096 pairs with 128 MLP neurons. Real bottleneck is $d_{\text{model}}=32$.

R1.5 (Q2): MLP ablation barely hurts — most memorisation is linear.



1.4b MLP activations: dense, distributed.



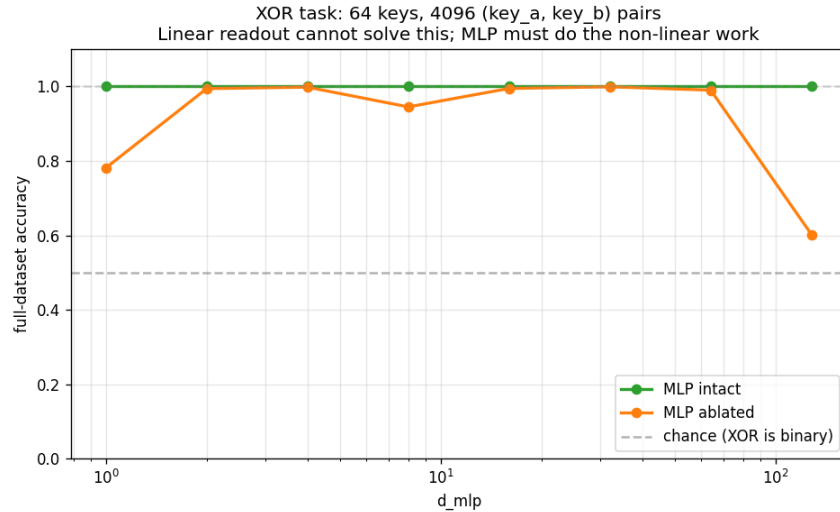
1.5 Ablation: 1.000 → 0.826. MLP contributes ~17 points only.

Q2 answer: not CiS. Bulk of memorisation lives in embed → attention → unembed; MLP non-linearity is not load-bearing.

[RED-TEAM] 17 points is non-zero, so the MLP does *something*. But it is clearly not the locus of memorisation.

R1.6 (Q3): An obvious CiS upgrade — and a red-team that breaks it.

Task: input = [BOS, key_a, key_b], target = $v_a \text{ XOR } v_b$. 64 keys, 4096 input pairs.



1.6 XOR task: with MLP intact all d_mlp solve it; with MLP ablated still $\geq 99\%$.

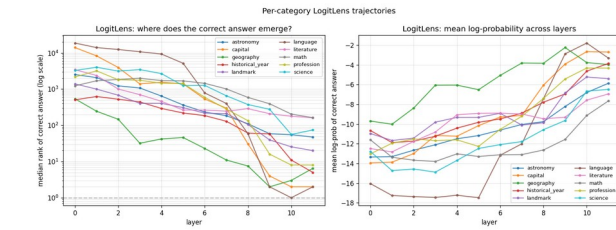
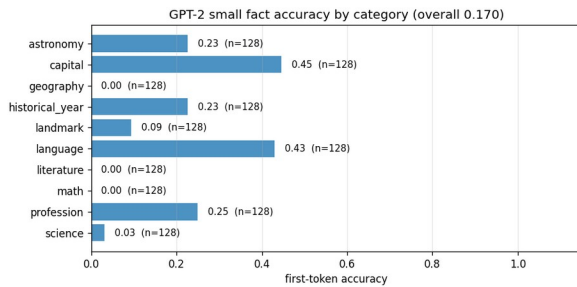
Q3 answer: multi-token input + non-linear target still does not isolate the MLP.

[RED-TEAM] The softmax inside attention is itself non-linear; a 1-layer attention can implement XOR-like routing on its own. To isolate MLP-level CiS, constrain the attention layer (linear/no-softmax attention, frozen attention pattern, or single-token multi-feature input).

GPT-2 memorised facts

Setup: 1280 prompts $\times$ 10 categories on GPT-2 small. First-token accuracy. LogitLens on resid_post at every layer.

R2.1 (Q1) + R2.2 (Q2): Easy facts known; emergence smooth across layers 6-10.

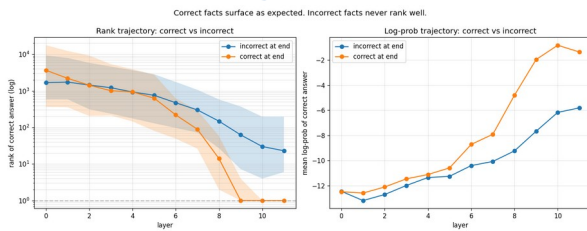


2.2 LogitLens median rank + mean log-prob, per category.

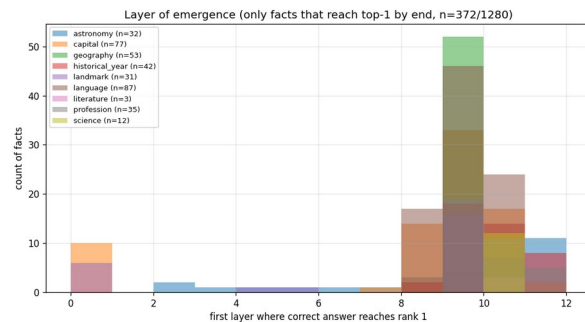
2.1 Accuracy by category (overall 17%).

Capitals 44%, languages 43%, professions 25%. Geography / literature / math at 0% are almost certainly first-token-metric artefacts. Emergence not localised to one layer; it builds gradually from mid-network onward.

R2.3 / R2.4: Wrong facts never rank well; median first-top-1 layer is 9 of 12.



2.3 Correct vs incorrect facts: rank trajectory.



2.4 Layer at which correct answer first reaches rank 1.

Incorrect-at-end facts hover at rank > 50 across all layers these are not 'known but suppressed', the model genuinely does not know them at this scale.

[RED-TEAM] LogitLens projects through ln_final + unembed which is not what later layers actually do. Smoothness reflects projection overlap, not necessarily causal contribution.

R2.5 (Q3) and R2.6 (Q4): Confounders, and how to do this better.

Confounders (rough severity order):

1) First-token-only metric (kills 0% categories), 2) multiple valid continuations, 3) tokenisation of multi-token answers, 4) identity-mapping (subject == answer), 5) domain bias toward capitals, 6) LogitLens is an approximation.

Better methods (what I'd run next):

Activation patching (causal localisation), linear probes per layer, logit-difference vs distractor, train SAEs at each layer and look for fact features.

Takeaways

Task 1 is not CiS most memorisation is linear; MLP contributes ~17 points only. • A natural CiS upgrade (XOR over multi-token input) still fails to isolate the MLP because attention's softmax is non-linear too. Clean MLP-level CiS needs a constrained attention layer. • GPT-2 small knows easy facts (capitals, languages ~44%); emergence is smooth across mid-to-late layers, not localised. • The biggest single fix for Task 2 would be filtering to clean single-token answers and adding a small activation-patching study.